

0.7. La boucle `do ... while` : au moins un tour

La boucle qui vérifie sa condition à la fin : elle fait toujours au moins un tour.

1. Ce qu'il faut comprendre

`do { ... } while (i < 20);` fait le tour **d'abord**, et vérifie la condition **ensuite**. Il fait donc **toujours au moins un tour**, même si la condition est fausse dès le départ.

Ici, le résultat est le même qu'avec `for` et `while` : les trois boucles savent faire la même chose. On choisit celle qui se lit le mieux.

Attention au **point-virgule** après `while (i < 20)` : il est obligatoire avec `do`, et seulement avec lui.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction `poser\(\)` — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **887 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         40 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AdresseCase    10 o ← poser() - trouve l'adresse d'une case de l'écran
CopierTuiles   18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser() × 1    18 o sur place, dans main() + AdresseCase (10 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

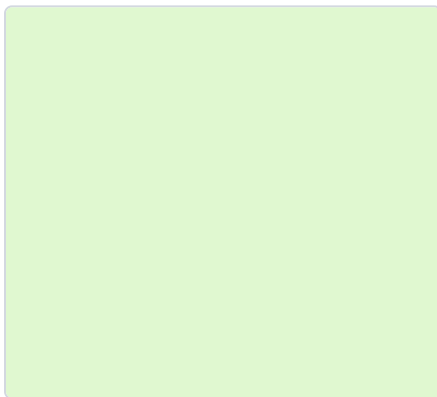
```
int main() {                                // Le programme commence ici.
    uint8_t i = 0;                          // Une variable i : le numéro de la lettre ET de la colonne.
                                           // uint8_t = un nombre entier de 0 à 255. On part de 0.

    do {                                    // do = « fais » : le bloc est fait AU MOINS une fois...
        poser(i, 0, ALPHABET[i]);          // la lettre n° i, dans la colonne i
                                           // (ALPHABET[0] = A, ALPHABET[1] = B, ...)
        i++;                               // i gagne 1 : la colonne et la lettre suivantes.
    } while (i < 20);                      // la condition est vérifiée À LA FIN :
                                           // ... puis recommencé tant que i < 20.
                                           // Quand i vaut 20 (après le T), on sort.

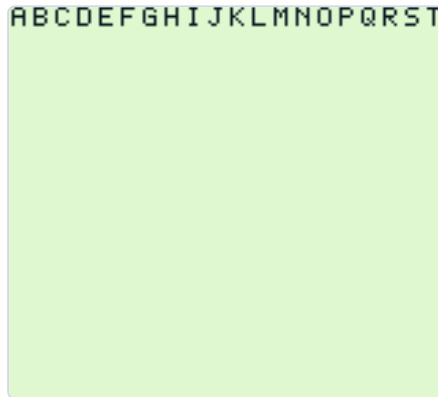
    while (true) {                         // La boucle de jeu : recommence pour toujours...
        image();                           // ... attendre l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

A à T sur toute la première ligne, comme au 0.5.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la ligne 0 est pleine : A à T
- ✓ rien ne déborde sur la ligne 1